Source Code - XOR Arbiter PUF (Verilog)

```
`timescale 1ns/1ps

// ============================================================
// TOP MODULE
// ============================================================

module xor_apuf (
    input  wire clk,
    input  wire reset,
    output wire final_response
);

    // ----------------------------------------------------------
    // Challenge
    // ----------------------------------------------------------

    reg [63:0] challenge;

    always @(posedge clk or posedge reset) begin
        if (reset)
            challenge <= 64'hACE11234ABCD5678;
        else
            challenge <= {
                challenge[62:0],
                challenge[63] ^ challenge[62]
            };
    end


    // ----------------------------------------------------------
    // Challenge permutation
    // ----------------------------------------------------------

    wire [63:0] perm;

    assign perm = {
        challenge[3],  challenge[7],  challenge[12], challenge[20],
        challenge[1],  challenge[9],  challenge[15], challenge[22],
        challenge[0],  challenge[18], challenge[6],  challenge[25],
        challenge[30], challenge[10], challenge[11], challenge[19],
        challenge[4],  challenge[8],  challenge[16], challenge[28],
        challenge[2],  challenge[5],  challenge[13], challenge[21],
        challenge[23], challenge[24], challenge[26], challenge[27],
        challenge[29], challenge[31], challenge[32], challenge[33],
        challenge[34], challenge[35], challenge[36], challenge[37],
        challenge[38], challenge[39], challenge[40], challenge[41],
        challenge[42], challenge[43], challenge[44], challenge[45],
        challenge[46], challenge[47], challenge[48], challenge[49],
        challenge[50], challenge[51], challenge[52], challenge[53],
        challenge[54], challenge[55], challenge[56], challenge[57],
        challenge[58], challenge[59], challenge[60], challenge[61],
        challenge[62], challenge[63], challenge[14], challenge[17]
    };


    // ----------------------------------------------------------
    // Three Arbiter PUFs
    // ----------------------------------------------------------

    wire response1;
    wire response2;
    wire response3;

    arbiter_puf_64 PUF1 (
        .challenge(perm),
        .response(response1)
    );

    arbiter_puf_64 PUF2 (
        .challenge({perm[62:0], perm[63]}),
```

```
        .response(response2)
    );

    arbiter_puf_64 PUF3 (
        .challenge({perm[1:0], perm[63:2]}),
        .response(response3)
    );


    // ------------------------------------------------------
    // Response shift registers
    // ------------------------------------------------------

    reg [14:0] sr1;
    reg [14:0] sr2;
    reg [14:0] sr3;

    always @(posedge clk or posedge reset) begin

        if (reset) begin
            sr1 <= 15'b0;
            sr2 <= 15'b0;
            sr3 <= 15'b0;
        end
        else begin
            sr1 <= {sr1[13:0], response1};
            sr2 <= {sr2[13:0], response2};
            sr3 <= {sr3[13:0], response3};
        end

    end


    // ------------------------------------------------------
    // TMV / parity
    // ------------------------------------------------------

    wire g1;
    wire g2;
    wire g3;

    assign g1 = ^sr1;
    assign g2 = ^sr2;
    assign g3 = ^sr3;


    // ------------------------------------------------------
    // Final XOR response
    // ------------------------------------------------------

    assign final_response = g1 ^ g2 ^ g3;

endmodule



// ============================================================
// 64-STAGE ARBITER PUF
// ============================================================

module arbiter_puf_64 (

    input wire [63:0] challenge,
    output wire response

);

    wire [64:0] path_a;
    wire [64:0] path_b;

    // Start two paths differently for functional RTL simulation
    assign path_a[0] = 1'b1;
    assign path_b[0] = 1'b0;
```

```
    genvar i;

    generate

        for (i = 0; i < 64; i = i + 1) begin : MUX_STAGE

            puf_mux_stage STAGE (
                .a_in(path_a[i]),
                .b_in(path_b[i]),
                .select(challenge[i]),
                .a_out(path_a[i+1]),
                .b_out(path_b[i+1])
            );

        end

    endgenerate

    // Functional arbiter
    assign response = path_a[64] ^ path_b[64];

endmodule



// ============================================================
// PUF MUX STAGE
// ============================================================

module puf_mux_stage (

    input wire a_in,
    input wire b_in,
    input wire select,

    output wire a_out,
    output wire b_out

);

    assign a_out = select ? b_in : a_in;

    assign b_out = select ? a_in : b_in;

endmodule
```